A Mathematical Derivation

We now derive the update equations from the ELBO maximization rules in Equation 10.

Let $\mathbf{A}$ be a tensor network defined by the set of the nodes $\{\mathbf{A}^1, \mathbf{A}^2, \ldots, \mathbf{A}^n\}$ and edges $\{\boldsymbol{e}^1, \ldots, \boldsymbol{e}^m\}$.

We firstly derive the update equation for the parameters of a single node $\mathbf{A}^i$. It is obtained by solving

$$\log p(\mathbf{A}^i) = \mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\boldsymbol{y}, \Theta)]. \tag{20}$$

We explicit the two sides of the equation, considering that in the right side, we can ignore all the terms that would be constant in respect to $\mathbf{A}^i$.

$$\mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\boldsymbol{y}, \Theta)] = \sum_s \mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log l_s] + \mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\mathbf{A}^i)]. \tag{21}$$

Note that the arguments of the gaussian are kept as tensors, the distribution is the same as if they would be vectorized, calculated the logarithm of the distribution and reshaped in the original form.

Except a constant the right side takes the form:

$$\log l_s \simeq -\frac{\tau}{2}(y_s - \langle \mathbf{A}\mathbf{X}_s \rangle)^2 = -\frac{\tau}{2}y_s^2 + \tau y_s \langle \mathbf{A}\mathbf{X}_s \rangle - \frac{\tau}{2} \langle \mathbf{X}_s\mathbf{A} \rangle \langle \mathbf{A}\mathbf{X}_s \rangle \tag{22}$$

$$= -\frac{\tau}{2}y_s^2 + \tau y_s \left\langle \hat{\mathbf{A}}^i\mathbf{A}^i\mathbf{X}_s \right\rangle - \frac{\tau}{2} \left\langle \mathbf{X}_s\mathbf{A}^i\hat{\mathbf{A}}^i\hat{\mathbf{A}}^i\mathbf{A}^i\mathbf{X}_s \right\rangle, \tag{23}$$

$$\log p(\mathbf{A}^i) = -\frac{1}{2}\mathbf{A}^i\tilde{\mathbf{\Lambda}}^{E(i)}\mathbf{A}^i + \text{constant}. \tag{24}$$

Where in step 23 separate the tensor network $\mathbf{A} = \hat{\mathbf{A}}^i\mathbf{A}^i$, as a contraction between the environment of the node and the node $\mathbf{A}^i$ itself. The term $-\frac{\tau}{2}y_s^2$ is a constant and so it can be removed. For brevity the expectation will be written $\mathbb{E}$ without the subscript, which is often obvious by the context.

$$\mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log l_s] = y_s\mathbb{E}[\tau] \left\langle \mathbb{E}[\hat{\mathbf{A}}^i]\mathbf{A}^i\mathbf{X}_s \right\rangle - \frac{\mathbb{E}[\tau]}{2} \left\langle \mathbf{A}^i\mathbf{X}_s\mathbb{E}[\hat{\mathbf{A}}^i \otimes \hat{\mathbf{A}}^i]\mathbf{X}_s\mathbf{A}^i \right\rangle \tag{25}$$

$$= y_s\mathbb{E}[\tau] \left\langle \hat{\boldsymbol{\mu}}^i\mathbf{A}^i\mathbf{X}_s \right\rangle - \frac{\mathbb{E}[\tau]}{2} \left\langle \mathbf{A}^i\mathbf{X}_s\hat{\mathbf{P}}^i\mathbf{X}_s\mathbf{A}^i \right\rangle, \tag{26}$$

$$\mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\mathbf{A}^i)] = -\frac{1}{2}\mathbf{A}^i\mathbb{E}[\tilde{\mathbf{\Lambda}}^{E(i)}]\mathbf{A}^i = -\frac{1}{2}\mathbf{A}^i\tilde{\mathbf{\Gamma}}^{E(i)}\mathbf{A}^i. \tag{27}$$

Where we used the definition of $\mathbf{\Gamma}$ in Equation 12 and use the diagonalization $\tilde{\mathbf{\Gamma}} = diag(\mathbf{\Gamma})$.

Finally

$$\sum_s \mathbb{E}[\log l_s] + \mathbb{E}[\log p(\mathbf{A}^i)] = \tag{28}$$

$$= \sum_s y_s\mathbb{E}[\tau] \left\langle \hat{\boldsymbol{\mu}}^i\mathbf{A}^i\mathbf{X}_s \right\rangle - \frac{\mathbb{E}[\tau]}{2} \left\langle \mathbf{A}^i\mathbf{X}_s\hat{\mathbf{P}}^i\mathbf{X}_s\mathbf{A}^i \right\rangle - \frac{1}{2} \left\langle \mathbf{A}^i\tilde{\mathbf{\Gamma}}^{E(i)}\mathbf{A}^i \right\rangle = \tag{29}$$

$$= \left\{ \mathbb{E}[\tau] \sum_s y_s\hat{\boldsymbol{\mu}}^i\mathbf{X}_s \right\} \mathbf{A}^i - \frac{1}{2}\mathbf{A}^i \left\{ \mathbb{E}[\tau] \sum_s \mathbf{X}_s\hat{\mathbf{P}}^i\mathbf{X}_s + \tilde{\mathbf{\Gamma}}^{E(i)} \right\} \mathbf{A}^i. \tag{30}$$

If we consider a generic gaussian density distribution of $\mathbf{A}^i$, we can notice that the log of the density has the form $\{{\mathbf{\Sigma}^i}^{-1}\boldsymbol{\mu}^i\}\mathbf{A}^i - \frac{1}{2}\mathbf{A}^i\{{\mathbf{\Sigma}^i}^{-1}\}\mathbf{A}^i$. Obtaining the updates in Equation 13.

For the edges instead we need to repeat the same process, substituting the edge probability distribution and solving the equation:

$$\log p(\boldsymbol{e}^j) = \mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_j)}[\log p(\boldsymbol{y}, \Theta)]. \tag{31}$$

The terms we consider are

$$\mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_j)}[\log p(\boldsymbol{y}, \Theta)] = \sum_{i \in N(j)} \mathbb{E}[\log p(\mathbf{A}^i)] + \mathbb{E}[\log p(\boldsymbol{e}^j)]. \tag{32}$$

Note that $\tilde{\mathbf{\Lambda}}^{E(i)}$ is diagonal. The diagonal elements are the parameters associated to the edges of node $\mathbf{A}^i$. The determinant of $\tilde{\mathbf{\Lambda}}^{E(i)}$ is then

$$\left|\tilde{\mathbf{\Lambda}}^{E(i)}\right| = \prod_{\boldsymbol{e}^j \in E(i)} \prod_{k=1}^{\dim(\boldsymbol{e}^j)} \lambda_{jk}^{D_{ij}}, \tag{33}$$

where $D_{ij} = \prod_{\boldsymbol{e}^k \in E(i) \wedge \boldsymbol{e}^k \neq \boldsymbol{e}^j} \dim(\boldsymbol{e}^k)$.

The logarithm of the prior becomes a series of sums.

$$\log p(\boldsymbol{y}, \Theta) = \log p(\tau) + \sum_s \log l_s + \sum_{i \in \text{nodes}} \log p(\mathbf{A}^i) + \sum_{j \in \text{edges}} p(\boldsymbol{e}^j), \tag{34}$$

where we have

$$\mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_j)}\left[\log p\left(\mathbf{A}^i\right)\right] = \tag{35}$$

$$= \frac{1}{2} D_{ij} \sum_{k=1}^{\dim\left(\boldsymbol{e}^j\right)} \log \lambda_{jk} - \frac{1}{2}\mathbb{E}\left[vec\left(\mathbf{A}^i\right)^T \tilde{\mathbf{\Lambda}}^{E(i)} vec\left(\mathbf{A}^i\right)\right] = \tag{36}$$

$$= \frac{1}{2} D_{ij} \left\langle \log\left(\boldsymbol{\lambda}_j\right) \mathbf{1}\right\rangle - \frac{1}{2}\left\langle tr_{E(i)/j}\left(\mathbb{E}\left[\mathbf{A}^i \otimes \mathbf{A}^i\right] \tilde{\mathbf{\Gamma}}^{E(i)/j}\right)_{\boldsymbol{e}^j \boldsymbol{e}^j} \boldsymbol{\lambda}_j\right\rangle, \tag{37}$$

$$\mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_j)}\left[\log p\left(\boldsymbol{e}^j\right)\right] = \left\langle \boldsymbol{\alpha}_j^p \log\left(\boldsymbol{\lambda}_j\right)\right\rangle - \left\langle \mathbf{1} \log\left(\boldsymbol{\lambda}_j\right)\right\rangle - \left\langle \boldsymbol{\beta}_j^p \boldsymbol{\lambda}_j\right\rangle. \tag{38}$$

Summing both term we finally obtain:

$$\sum_{i \in N(j)} \mathbb{E}\left[\log p\left(\mathbf{A}^i\right)\right] + \mathbb{E}\left[\log p\left(\boldsymbol{e}^j\right)\right] = \tag{39}$$

$$= \left\langle \boldsymbol{\alpha}_j^p \log\left(\boldsymbol{\lambda}_j\right)\right\rangle - \left\langle \boldsymbol{\beta}_j^p \boldsymbol{\lambda}_j\right\rangle - \left\langle \mathbf{1} \log\left(\boldsymbol{\lambda}_j\right)\right\rangle - \tag{40}$$

$$- \frac{1}{2} \sum_{i \in N(j)} \left\langle tr_{E(i)/j}\left(\mathbb{E}\left[\mathbf{A}^i \otimes \mathbf{A}^i\right] \tilde{\mathbf{\Gamma}}^{E(i)/j}\right)_{\boldsymbol{e}^j \boldsymbol{e}^j} \boldsymbol{\lambda}_j\right\rangle + \frac{1}{2} D_{ij} \left\langle \mathbf{1} \log\left(\boldsymbol{\lambda}_j\right)\right\rangle = \tag{41}$$

$$= \left\langle \left\{ \boldsymbol{\alpha}_j^p + \frac{1}{2} \sum_{i \in N(j)} D_{ij} \mathbf{1} - \mathbf{1} \right\} \log\left(\boldsymbol{\lambda}_j\right) \right\rangle - \left\langle \left\{ \beta_j^p + \frac{1}{2} \sum_{i \in N(j)} tr_{E(i)/j}\left(\left[\mathbf{\Sigma}^i + \boldsymbol{\mu}^i \otimes \boldsymbol{\mu}^i\right] \tilde{\mathbf{\Gamma}}^{E(i)/j}\right)_{\boldsymbol{e}^j \boldsymbol{e}^j} \right\} \boldsymbol{\lambda}_j \right\rangle. \tag{42}$$

As previously, considering the log form of the gamma distribution:

$$\log(Ga(x,\alpha,\beta)) \sim \{(\alpha-1)\}\log x - \{\beta\}x, \tag{43}$$

we can identified in vectorized form the new parameters in equation 14.

In line 36, to easily transition the derivation, we write for a moment the equations in vectorized form. The passage in line 36 can be better derived if we recall the definition of $\tilde{\boldsymbol{\Lambda}}^{E(i)}$ which is the diagonal matrix, with diagonal the outer product of the edge parameters vectorized.

$$diag(\tilde{\boldsymbol{\Lambda}}^{E(i)}) = vec\left(\bigotimes_{j\in E(i)} \boldsymbol{\lambda}_j\right), \tag{44}$$

$$vec(\mathbf{A}^i)^T\tilde{\boldsymbol{\Lambda}}^{E(i)}vec\left(\mathbf{A}^i\right) = tr\left(vec(\mathbf{A}^i)^T\tilde{\boldsymbol{\Lambda}}^{E(i)}vec\left(\mathbf{A}^i\right)\right) = tr\left(vec\left(\mathbf{A}^i\right)vec(\mathbf{A}^i)^T\tilde{\boldsymbol{\Lambda}}^{E(i)}\right). \tag{45}$$

In order to provide a concrete example of the substitution of the expactation of Equation 45 in line 36 we consider a three dimensional node $\mathbf{A}^i$. We have to be explicit in edges association with label, and assume that $\bar{E}(i) = \{\boldsymbol{e}^1, \boldsymbol{e}^2, \boldsymbol{e}^3\}$ and label the element of the vector with the indices

$$(l,m,n) \rightarrow (\boldsymbol{e}^1, \boldsymbol{e}^2, \boldsymbol{e}^3).$$

$$tr(vec(\mathbf{A}^i)vec(\mathbf{A}^i)^T\tilde{\boldsymbol{\Lambda}}^{E(i)}) = \tag{46}$$

$$= \sum_{lmn}\sum_{l'm'n'}\sum_{l''m''n''} \mathbf{A}^i_{lmn}\mathbf{A}^i_{l'm'n'}\delta_{l',l''}\delta_{m',m''}\delta_{n',n''}\lambda_{1l''}\lambda_{2m''}\lambda_{3n''}\delta_{l,l''}\delta_{m,m''}\delta_{n,n''} \tag{47}$$

$$= \sum_{lmn}\sum_{l'm'n'} \mathbf{A}^i_{lmn}\mathbf{A}^i_{l'm'n'}\delta_{l,l'}\delta_{m,m'}\delta_{n,n'}\lambda_{1l}\lambda_{2m}\lambda_{3n}. \tag{48}$$

Remembering that

$$\tilde{\boldsymbol{\Gamma}}^{E(i)} = \bigotimes_{j\in E(i)} \mathbb{E}[diag(\boldsymbol{\lambda}_j)], \tag{49}$$

now the expectation respect to the parameters $\boldsymbol{\lambda}_3$ associated to edge $\boldsymbol{e}^3$ can be calculated as:

$$\mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_3)}[tr(vec(\mathbf{A}^i)vec(\mathbf{A}^i)^T\tilde{\boldsymbol{\Lambda}}^{E(i)})] = \tag{50}$$

$$= \mathbb{E}\left[\sum_{lmn}\sum_{l'm'n'} \mathbf{A}^i_{lmn}\mathbf{A}^i_{l'm'n'}\delta_{l,l'}\delta_{m,m'}\delta_{n,n'}\lambda_{1l}\lambda_{2m}\lambda_{3n}\right] = \tag{51}$$

$$= \sum_{nn'}\delta_{n,n'}\sum_{lm}\sum_{l'm'}\mathbb{E}\left[\mathbf{A}^i_{lmn}\mathbf{A}^i_{l'm'n'}\right]\mathbb{E}\left[\delta_{l,l'}\delta_{m,m'}\lambda_{1l}\lambda_{2m}\right]\lambda_{3n} = \tag{52}$$

$$= \sum_{nn'}\delta_{n,n'}\, tr_{E(i)/3}\left(\mathbb{E}\left[\mathbf{A}^i\otimes\mathbf{A}^i\right]\tilde{\boldsymbol{\Gamma}}^{E(i)/3}\right)_{nn'}\lambda_{3n} = \tag{53}$$

$$= \sum_{n} tr_{E(i)/3}\left(\mathbb{E}\left[\mathbf{A}^i\otimes\mathbf{A}^i\right]\tilde{\boldsymbol{\Gamma}}^{E(i)/3}\right)_{nn}\lambda_{3n} = \tag{54}$$

$$= \left\langle tr_{E(i)/3}\left(\mathbb{E}\left[\mathbf{A}^i\otimes\mathbf{A}^i\right]\tilde{\boldsymbol{\Gamma}}^{E(i)/3}\right)_{\boldsymbol{e}^3,\boldsymbol{e}^3}\boldsymbol{\lambda}_3\right\rangle. \tag{55}$$

Where we have indexed with $\{\boldsymbol{e}^3, \boldsymbol{e}^3\}$ the diagonal elements of the trace in the edge 3. To be more specific, due the contraction inside the trace involving all edges of node i twice from the outer product of $\mathbf{A}^i$ and all the edges twice except edge 3 given by the $\tilde{\mathbf{\Gamma}}$ element, the result of the trace will be just a bidimensional tensor in the space of $\boldsymbol{e}^3 \otimes \boldsymbol{e}^3$. We could have indeed simply write that we take the diagonal of the result, but we add the edge as index to emphasise that it lives in the space spanned by the edge of which we do not compute the expectation.

Now onto the τ parameters, we need to consider the following terms:

$$\mathbb{E}_{q(\Theta/\tau)}[\log p(\boldsymbol{y}, \Theta)] = \sum_{s}^{S} \mathbb{E}[\log l_s] + \mathbb{E}[\log p(\tau)]. \tag{56}$$

$$\begin{aligned}
\mathbb{E}_{q(\Theta/\tau)}[\log l_s] &= \frac{1}{2}\log\tau - \frac{\tau}{2}\mathbb{E}[(y_s - \langle \mathbf{A}\mathbf{X}_s\rangle)^2] = & (57)\\
&= \frac{1}{2}\log\tau - \frac{\tau}{2}y_s^2 + \tau y_s \mathbb{E}[\langle \mathbf{A}\mathbf{X}_s\rangle] - \frac{\tau}{2}\mathbb{E}[\langle \mathbf{X}_s\mathbf{A}\rangle \langle \mathbf{A}\mathbf{X}_s\rangle] = & (58)\\
&= \frac{1}{2}\log\tau - \frac{\tau}{2}y_s^2 + \tau y_s \langle \mathbb{E}[\mathbf{A}]\mathbf{X}_s\rangle - \frac{\tau}{2}\langle \mathbb{E}[\mathbf{A}\otimes\mathbf{A}]\,(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle = & (59)\\
&= \frac{1}{2}\log\tau - \frac{\tau}{2}y_s^2 + \tau y_s \langle \boldsymbol{\mu}\mathbf{X}_s\rangle - \frac{\tau}{2}\langle (\mathbf{P} - \boldsymbol{\mu}\otimes\boldsymbol{\mu} + \boldsymbol{\mu}\otimes\boldsymbol{\mu})\,(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle = & (60)\\
&= \frac{1}{2}\log\tau - \frac{\tau}{2}y_s^2 + \tau y_s \langle \boldsymbol{\mu}\mathbf{X}_s\rangle - \frac{\tau}{2}\langle (\mathbf{P} - \boldsymbol{\mu}\otimes\boldsymbol{\mu})\,(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle - \frac{\tau}{2}\langle (\boldsymbol{\mu}\otimes\boldsymbol{\mu})\,(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle = & (61)\\
&= \frac{1}{2}\log\tau - \frac{\tau}{2}\left[y_s^2 - 2y_s\langle \boldsymbol{\mu}\mathbf{X}_s\rangle + \langle \boldsymbol{\mu}\mathbf{X}_s\rangle^2 + \langle (\mathbf{P} - \boldsymbol{\mu}\otimes\boldsymbol{\mu})\,(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle\right] = & (62)\\
&= \frac{1}{2}\log\tau - \frac{\tau}{2}\left[(y_s - \boldsymbol{\mu}\mathbf{X}_s)^2 + \langle (\mathbf{P} - \boldsymbol{\mu}\otimes\boldsymbol{\mu})\,(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle\right], & (63)
\end{aligned}$$

$$\mathbb{E}_{q(\Theta/\tau)}[\log p(\tau)] = (\alpha_\tau^p - 1)\log\tau - \beta_\tau^p\tau. \tag{64}$$

Summing the previous terms we obtain

$$\begin{aligned}
&\sum_{s=1}^{S}\mathbb{E}_{q(\Theta/\tau)}[\log l_s] + \mathbb{E}_{q(\Theta/\tau)}[\log p\,(\tau)] = & (65)\\
&= \sum_{s=1}^{S}\frac{1}{2}\log\tau - \frac{\tau}{2}\left(\|y_s - \boldsymbol{\mu}\mathbf{X}_s\|^2 + \langle (\mathbf{P} - \boldsymbol{\mu}\otimes\boldsymbol{\mu})\,(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle\right) + (\alpha_\tau^p - 1)\log\tau - \beta_\tau^p\tau = & (66)\\
&= \left[\frac{S}{2} + \alpha_\tau^p - 1\right]\log\tau - \tau\left[\beta_\tau^p + \frac{1}{2}\sum_{s=1}^{S}\|y_s - \boldsymbol{\mu}\mathbf{X}_s\|^2 + \langle (\mathbf{P} - \boldsymbol{\mu}\otimes\boldsymbol{\mu})\,(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle\right]. & (67)
\end{aligned}$$

And obtaining equation equation 15 using the usual moment matching for gamma distribution.

B Complexity of the node/edge/noise updates.

For the complexity analysis, we consider a reshaping in matrices and vector, this is because otherwise the precise cost of contraction is dependent on the exact shape and size of the tensor. That said we will consider that the contraction will be well optimized, as is fair when using ad-hoc studied python libraries. We can then just use the same computational cost that would happen if the tensors were matricized.

Given a generic node $\mathbf{A}^i$ with edge set $E(i)$, each edge dimensions $d_j = \dim(\boldsymbol{e}^j) \quad \forall j \in E(i)$ and total (vectorized) size:

$$R_i = \prod_{j\in E(i)} d_j.$$

The variational parameters attached to the node are the mean $\boldsymbol{\mu}^i \in \bigotimes_{j\in E(i)} V_j$ and the second moment $\mathbf{P}^i \in \bigotimes_{j\in E(i)}(V_j \otimes V_j)$, whose matricizations are of size R_i and $R_i \times R_i$ respectively, so storing a single node costs $\sim \mathcal{O}(R_i^2)$ memory.

The node update equation 13 requires:

- calculating and accumulating the data statistic $\sum_s \hat{\mathbf{P}}^i\, \mathbf{X}_s^P$ of the environment, assuming that when optimized we can consider all nodes contractions as having the same cost, dominated by the contraction of the sigma network. We can upperbound the cost of contracting a node to the environment to be given by $R_i \dim(\boldsymbol{e}^j)$ where $\boldsymbol{e}^j$ is the contracting edge. Now calling $R = \max(R_i)$ and $d = \max(\dim \boldsymbol{e}^j)$, the complexity of each node addition to the $\boldsymbol{\mu}$ network is upperbounded by Rd, which is repeated for $N-1$ nodes and S samples, obtaining $\mathcal{O}(S(N-1)\, Rd)$. For the $\mathbf{P}$ network the edges are contracted in couples, so both the node size and the contracting bond are squared, giving R^2d^2 per node addition and $\mathcal{O}(S(N-1)\, R^2d^2)$ overall. So the total cost is $\mathcal{O}(S(N-1)\,(R^2d^2+Rd))$, dominated by the $\mathbf{P}$ network term $\mathcal{O}(S(N-1)\, R^2d^2)$.
- adding the diagonal edge-precision term $\tilde{\boldsymbol{\Gamma}}^{E(i)}$, $\mathcal{O}(R_i)$
- inverting the resulting $R_i \times R_i$ precision to obtain $\boldsymbol{\Sigma}^{i\star}$, $\mathcal{O}(R_i^3)$
- the mean update $\boldsymbol{\mu}^{i\star} = \mathbb{E}[\tau]\, \boldsymbol{\Sigma}^{i\star} \sum_s y_s\, \hat{\boldsymbol{\mu}}^i \mathbf{X}_s^\mu$ is composed by a matrix-vector product $\mathcal{O}(R_i^2)$ plus an $\mathcal{O}(S\,(N-1)Rd)$ accumulation of the right-hand side.

The node update therefore costs

$$\mathcal{O}\big(S\,(N-1)(R^2d^2+Rd)+R^3\big) \quad \text{time}, \qquad \mathcal{O}\big(R^2\big) \quad \text{memory},$$

where the inversion of the single node dominates whenever $R \gtrsim S(N-1)d^2$, while for large sample sizes the $\mathbf{P}$ network accumulation $S(N-1)R^2d^2$ is the leading term.

The edge update equation 14 for edge j needs the partial trace $tr_{E(i)/j}\Big[\mathbf{P}^i \tilde{\boldsymbol{\Gamma}}^{E(i)/j}\Big]_{\boldsymbol{e}^j,\boldsymbol{e}^j}$, which contracts the second moment $\mathbf{P}^i$ against the diagonal precisions of the *other* edges of $\mathbf{A}^i$, with an optimal order this is $\mathcal{O}(R_i^2)$, while the concentration/rate $(\boldsymbol{\alpha}_j, \boldsymbol{\beta}_j)$ update is $\mathcal{O}(\dim(\boldsymbol{e}^j))$.

Summed over the edges of the node,

$$\sum_{\boldsymbol{e}^j \in E(i)} \mathcal{O}(R_i^2) = \mathcal{O}(|E(i)|\, R_i^2).$$

The noise update equation 15 contributes the residual $\sum_s \|y_s - \langle \boldsymbol{\mu} \mathbf{X}_s^\mu \rangle\|^2$ and the epistemic term $\sum_s \Big\langle (\mathbf{P} - \boldsymbol{\mu} \otimes \boldsymbol{\mu})\, \mathbf{X}_s^P \Big\rangle$; for node $\mathbf{A}^i$ the latter is again an $\mathcal{O}(S\, R_i^2)$ accumulation.

Hence a full sweep over the updates is dominated by $\mathcal{O}\big(S\,(N-1)(R^2d^2+Rd)+R^3\big)$, the node mean and covariance update.

C Uncertainty Quantification

A point prediction tells us where a model expects the target to be, but it says nothing about how much that expectation should be trusted. For Bayesian models we care equally about the predicted value and about the associated confidence. In this section we evaluate the quality of the predictive uncertainty produced by the BTNR method and compare it against the Bayesian baselines GP, BDE, HSBNN and BASS. We first define the metrics that quantify uncertainty quality, retrieved from Bayesian deep learning literature (Lakshminarayanan et al., 2017; Kuleshov et al., 2018; Tran et al., 2020), and then present the figures and tables that report the described metrics.

C.1 Predictive distribution and notation

Every model is assumed to output, for each test point $s \in \{1, \ldots, S\}$, a Gaussian predictive distribution

$$y_s \sim \mathcal{N}\big(\mu_s,\ \sigma_s^2\big), \tag{68}$$

where S is the number of test points, μ_s is the predictive mean and σ_s the predictive standard deviation. We write Φ and ϕ for the standard normal cumulative distribution and density functions, and $F_s(z) = \Phi\big((z-\mu_s)/\sigma_s\big)$ for the predictive cumulative distribution function at point s.

We evaluate ten regression tasks. Each combination of model and dataset is repeated over $K = 10$ random seeds.

For a scalar metric m with per seed values $\{m^{(1)}, \ldots, m^{(K)}\}$ we report the empirical mean and the sample standard deviation across seeds,

$$\bar{m} = \frac{1}{K}\sum_{k=1}^{K} m^{(k)}, \qquad \operatorname{std}(m) = \sqrt{\frac{1}{K-1}\sum_{k=1}^{K}\big(m^{(k)} - \bar{m}\big)^2}. \tag{69}$$

The BTNR entry in every figure and table is the best BTNR configuration for that dataset and metric.

C.2 Uncertainty metrics

Probabilistic accuracy. The negative log-likelihood (NLL) is the most widely reported scoring rule for regression uncertainty (Gal and Ghahramani, 2016; Lakshminarayanan et al., 2017). It measures how probable the observed targets are under the predictive distribution,

$$\mathrm{NLL} = \frac{1}{S}\sum_{s=1}^{S}\frac{1}{2}\left(\log\big(2\pi\sigma_s^2\big) + \frac{(y_s-\mu_s)^2}{\sigma_s^2}\right), \tag{70}$$

with lower values indicating a better fit. The continuous ranked probability score (CRPS) is a scoring rule introduced by Matheson and Winkler (1976) and widely adopted following the analysis of Gneiting and Raftery (2007). It measures the distance between the predictive cumulative distribution function and the empirical one,

$$\mathrm{CRPS} = \frac{1}{S}\sum_{s=1}^{S}\int_{-\infty}^{\infty}\big(F_s(z) - \mathbf{1}[\,z \geq y_s\,]\big)^2\,\mathrm{d}z. \tag{71}$$

For the Gaussian predictive distribution the per point integral has the closed form (Gneiting and Raftery, 2007)

$$\mathrm{CRPS}_s = \sigma_s\left[\frac{y_s-\mu_s}{\sigma_s}\Big(2\,\Phi\big(\tfrac{y_s-\mu_s}{\sigma_s}\big) - 1\Big) + 2\,\phi\big(\tfrac{y_s-\mu_s}{\sigma_s}\big) - \frac{1}{\sqrt{\pi}}\right]. \tag{72}$$

It is a proper scoring rule expressed in the units of y, more robust to outliers than NLL, and again lower is better. The results related to probabilistic accuracy are reported in Tables 2 and 3.

Calibration. Calibration tries to quantify whether the predicted uncertainty has the correct size (Guo et al., 2017; Kuleshov et al., 2018). The expected calibration error (ECE) was first proposed for classification through reliability estimates (Pakdaman Naeini et al., 2015; Guo et al., 2017) and later extended to regression by checking the coverage of predicted quantiles (Kuleshov et al., 2018). We fix a grid of quantile levels $P = \{0.1, 0.2, \ldots, 1.0\}$, where each level $p \in P$ is a nominal probability. For a given p the model is well calibrated if a fraction p of the targets falls below its predicted p-quantile. We define ECE in the main paper in 6 Equation 18. A value near zero means the model is well calibrated.

The prediction interval coverage probability (PICP) measures how often the targets fall inside the nominal 95% interval (Pearce et al., 2018),

$$\mathrm{PICP} = \frac{1}{S}\sum_{s=1}^{S}\mathbf{1}[\,\mu_s - z\,\sigma_s \leq y_s \leq \mu_s + z\,\sigma_s\,], \qquad z = \Phi^{-1}(0.975) \approx 1.96, \tag{73}$$

and is best when it equals its target of 0.95. The sharpness is the mean predicted standard deviation,

$$\mathrm{SH} = \frac{1}{S}\sum_{s=1}^{S} \sigma_s, \tag{74}$$

which captures how confident the model is: a smaller value means tighter predictive distributions.

Both PICP and sharpness are only meaningful once calibration is satisfied, since a model can be arbitrarily sharp while being badly miscalibrated. Results for calibration are shown in Figure 7, Figure 8 and Table 4.

Discrimination. Discrimination evaluates whether the uncertainty is effectively associating predictions by their error. We order the test points by predicted standard deviation in descending order and, for a removal fraction $r \in [0, 1]$, compute the root mean squared error on the points that remain,

$$\mathrm{RMSE}_{\mathrm{unc}}(r) = \sqrt{\frac{1}{|R_r|}\sum_{s\in R_r}(y_s - \mu_s)^2}, \qquad R_r = \{\text{points kept after removing the } \lceil rS \rceil \text{ most uncertain}\}. \tag{75}$$

The oracle curve $\mathrm{RMSE}_{\mathrm{oracle}}(r)$ performs the same operation but removes points in order of their true absolute error, giving the best achievable ordering. Normalising each curve by its value at $r = 0$, $\widehat{\mathrm{RMSE}}(r) = \mathrm{RMSE}(r)/\mathrm{RMSE}(0)$, the area under the sparsification error (AUSE) is the area between the uncertainty and oracle curves (Ilg et al., 2018),

$$\mathrm{AUSE} = \int_0^1 \left(\widehat{\mathrm{RMSE}}_{\mathrm{unc}}(r) - \widehat{\mathrm{RMSE}}_{\mathrm{oracle}}(r)\right) \mathrm{d}r, \tag{76}$$

where lower is better. As a complementary, measure the does not depend on thresholds, we also report the Spearman correlation ρ_{S} between the predicted standard deviations $\{\sigma_s\}$ and the absolute errors $\{|y_s - \mu_s|\}$. To compute it, each of the two quantities is sorted in ascending order and be indicated according to their position, from 1 for the smallest to S for the largest, ρ_{S} is then the ordinary Pearson correlation between these two position sequences rather than between the raw values. Results for discrimination are shown in Figure 6 and Table 5.

Outlier detection. These metrics are evaluated by inject synthetic outliers into the test set and probe whether the uncertainty can flag them as outliers, using the predictive standard deviation σ_s or the per point NLL as an anomaly score a_s (Hendrycks and Gimpel, 2017).

Treating the injected points as the positive class $y_s \in \{0, 1\}$, a threshold τ on the score induces a true positive rate, false positive rate and precision,

$$\mathrm{TPR}(\tau) = \frac{\sum_s y_s\, \mathbf{1}[a_s \geq \tau]}{\sum_s y_s}, \quad \mathrm{FPR}(\tau) = \frac{\sum_s (1 - y_s)\, \mathbf{1}[a_s \geq \tau]}{\sum_s (1 - y_s)}, \quad \mathrm{Prec}(\tau) = \frac{\sum_s y_s\, \mathbf{1}[a_s \geq \tau]}{\sum_s \mathbf{1}[a_s \geq \tau]}. \tag{77}$$

We report the area under the resulting ROC curve (AUROC, the area under TPR against FPR) and the area under the precision vs recall curve (AUPR, the area under Prec against TPR), both of which increase as detection improves. An uninformative score gives AUROC = 0.5. Outlier detection results are reported in Table 6.

Uncertainty decomposition. For BTNR we additionally separate the predictive standard deviation into an epistemic part σ_{epi}, which reflects reducible uncertainty due to limited data or model capacity, and an aleatoric part σ_{ale}, which reflects irreducible noise in the data (Kendall and Gal, 2017; Gal and Ghahramani, 2016). The scale free epistemic fraction summarises how much of the uncertainty is reducible,

$$\rho_{\mathrm{epi}} = \frac{\sigma_{\mathrm{epi}}}{\sigma_{\mathrm{epi}} + \sigma_{\mathrm{ale}}}, \tag{78}$$

where a large value signals uncertainty that could be reduced with more data or capacity and a small value signals inherent label noise. Uncertainty decomposition is shown for BTNR in Figure 9.

C.3 Uncertainty experiments

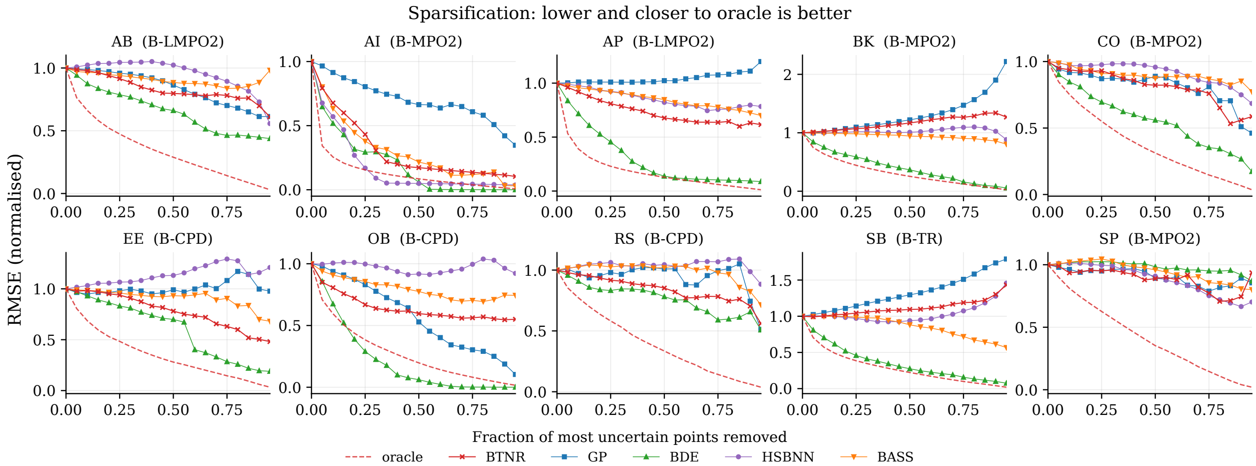

Figure 6: The plots address whether the uncertainty ranks points by error (Ilg et al., 2018). For each model we draw the normalised RMSE by uncertainty curve $\widehat{\mathrm{RMSE}}_{\mathrm{unc}}(r)$ against the removal fraction r, together with the oracle curve $\widehat{\mathrm{RMSE}}_{\mathrm{oracle}}(r)$, the area between them is the AUSE (see Equation 76). A good model produces an uncertainty curve that drops quickly and stays close to the oracle, and the BTNR entry in each panel is the configuration with the lowest AUSE for that dataset.

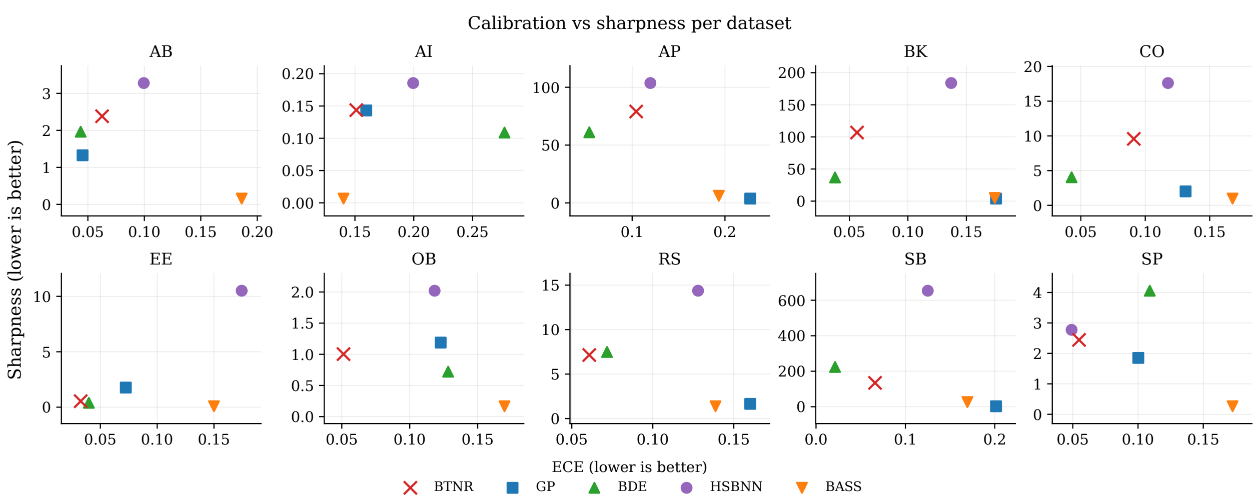

Figure 7: Each marker places a model at $x = \mathrm{ECE}$ (see Equation 18) and $y = \mathrm{SH}$ (see Equation 74), so the bottom left corner corresponds to the desirable combination of low calibration error and high confidence. The plot shows the raw pairs without normalisation, one panel per dataset. A model should be both calibrated and sharp, hence the bottom left corner (the origin) is the desirable combination of low calibration error and high confidence. For each dataset we report which tensor network structures is plotted.

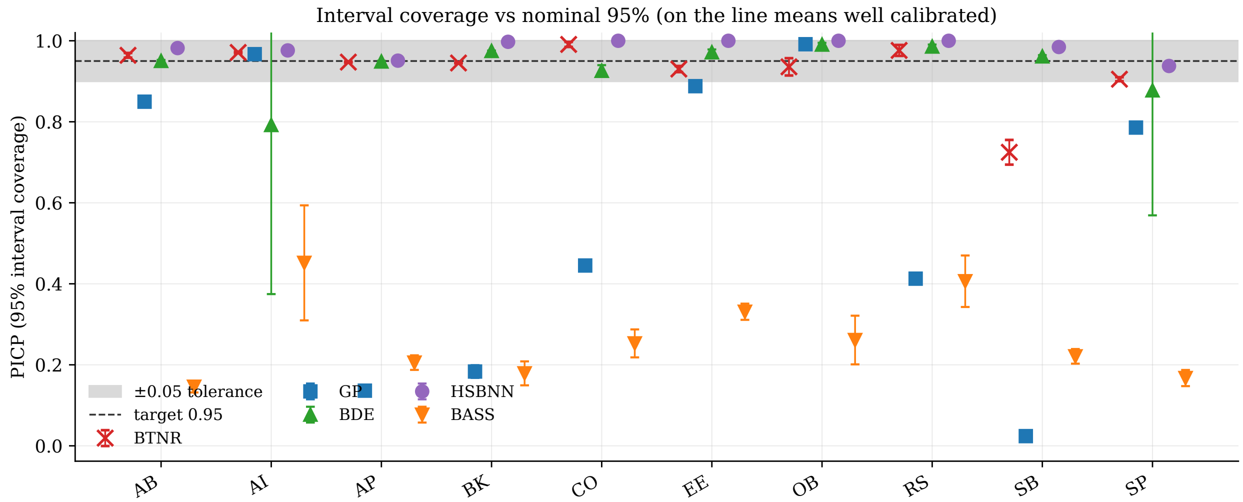

Figure 8: This figure checks whether the nominal 95% intervals contain 95% of the targets. For each model and dataset we plot the PICP (see Equation 73), with markers at the seed mean and error bars at the seed standard deviation. A dashed line marks the target of 0.95 inside a shaded tolerance band of ±0.05. Points on the line are well calibrated, points below indicate overconfidence and points above indicate underconfidence. The BTNR entry is the configuration whose PICP is closest to 0.95 for each dataset. The dashed line marks the target of 0.95 inside a shaded tolerance band of ±0.05.

Table 2: Test negative log-likelihood (NLL) (Equation 70) (lower is better), mean ± std. Best per dataset in bold. The table shows that BTNR attains competitive likelihoods across all ten tasks and remains stable even where some baselines diverge to large values

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
BTNR	2.15 ± 0.01	-0.65 ± 0.00	5.75 ± 0.01	6.05 ± 0.00	3.35 ± 0.08	0.82 ± 0.04	1.41 ± 0.04	3.19 ± 0.03	9.65 ± 0.41	2.48 ± 0.00
GP	2.43 ± 0.01	-0.67 ± 0.00	359.08 ± 7.04	244.53 ± 3.37	7.54 ± 0.00	2.24 ± 0.00	1.25 ± 0.01	9.04 ± 0.00	22967.24 ± 330.04	3.00 ± 0.00
BDE	**1.97** ± 0.01	**-2.51** ± 0.48	**5.03** ± 0.02	**4.55** ± 0.00	**2.74** ± 0.03	**0.44** ± 0.06	**-0.92** ± 0.22	**3.10** ± 0.03	**6.36** ± 0.03	2.62 ± 0.05
HSBNN	2.31 ± 0.01	-0.59 ± 0.00	5.98 ± 0.00	6.22 ± 0.02	3.87 ± 0.01	3.31 ± 0.01	1.73 ± 0.01	3.68 ± 0.01	7.55 ± 0.01	**2.45** ± 0.01
BASS	111.04 ± 28.15	81.98 ± 37.75	143.13 ± 23.30	96.12 ± 19.17	24.22 ± 4.54	19.05 ± 1.81	36.76 ± 21.93	13.37 ± 2.18	88.53 ± 16.50	85.71 ± 26.22

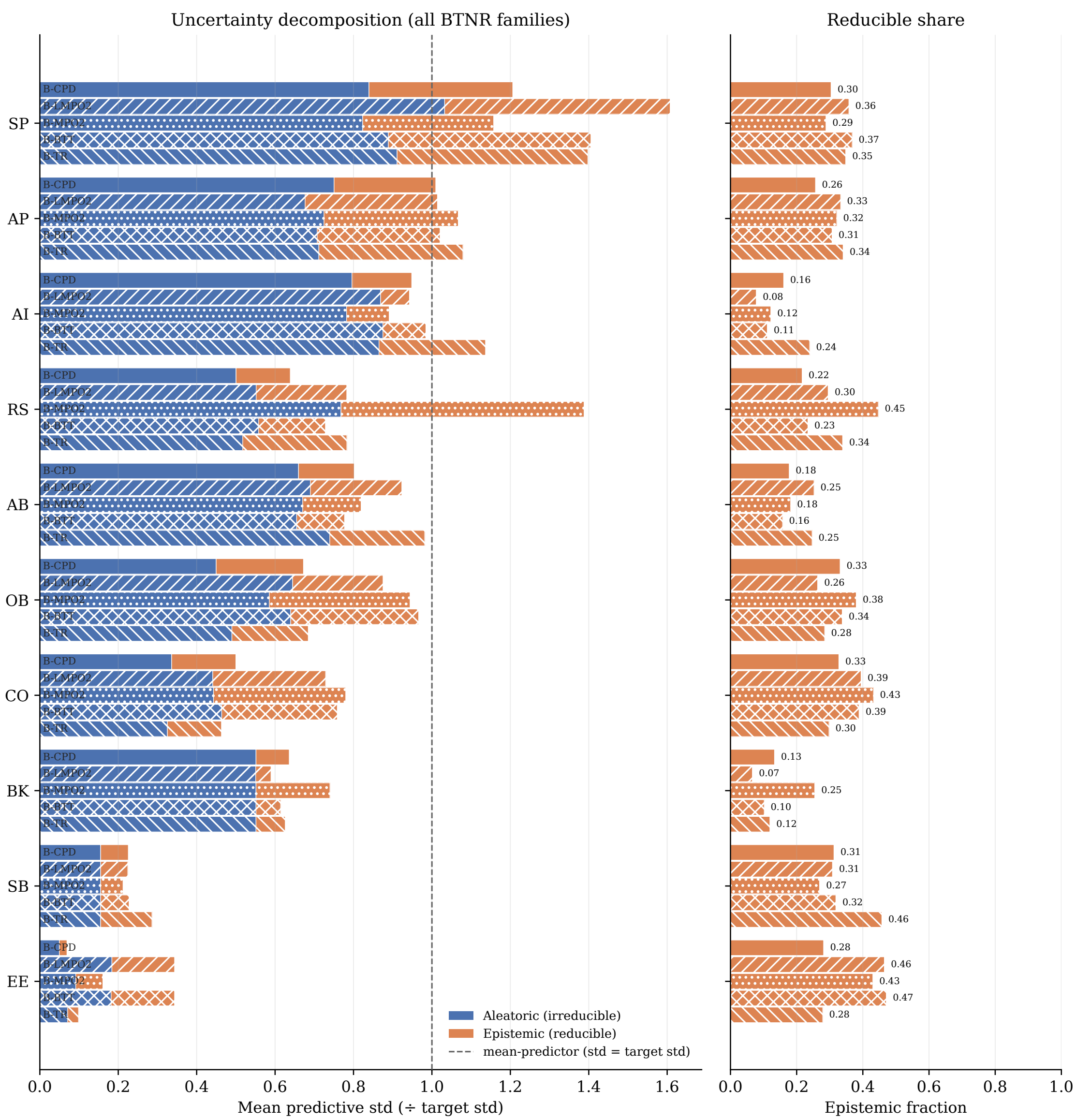

Figure 9: Uncertainty decomposition for BTNR. This figure, shows how much of the predictive uncertainty is reducible. Using the NLL best BTNR structure per dataset, the left panel stacks the mean aleatoric and epistemic standard deviations σ_{ale} and σ_{epi} in target units, with datasets sorted by their total. The right panel shows the scale free epistemic fraction ρ_{epi} (see Equation 78) defined above.

Table 3: Test continuous ranked probability score (CRPS) (Equation 71) (lower is better), mean ± std. Confirm the comparable results of BTNR in Table 2.

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
BTNR	1.11 ± 0.01	0.06 ± 0.00	37.10 ± 0.21	53.90 ± 0.07	3.58 ± 0.21	0.30 ± 0.01	0.56 ± 0.03	3.22 ± 0.12	181.94 ± 1.86	1.62 ± 0.00
GP	1.11 ± 0.00	0.05 ± 0.00	51.84 ± 0.61	47.19 ± 1.19	4.65 ± 0.00	1.17 ± 0.00	0.42 ± 0.00	4.27 ± 0.00	351.50 ± 1.68	1.90 ± 0.00
BDE	**1.03** ± 0.00	**0.04** ± 0.00	**32.12** ± 0.15	**17.90** ± 0.13	**2.37** ± 0.06	**0.21** ± 0.01	**0.27** ± 0.02	**2.98** ± 0.05	**116.64** ± 0.77	1.82 ± 0.07
HSBNN	1.21 ± 0.01	0.06 ± 0.00	45.98 ± 0.14	54.62 ± 1.98	5.30 ± 0.04	2.74 ± 0.03	0.64 ± 0.02	4.37 ± 0.05	221.10 ± 2.91	**1.59** ± 0.01
BASS	1.41 ± 0.01	**0.04** ± 0.00	50.69 ± 0.33	39.05 ± 1.42	4.24 ± 0.11	0.33 ± 0.00	0.80 ± 0.05	3.90 ± 0.14	189.85 ± 3.37	2.22 ± 0.03

Table 4: Expected calibration error (ECE) (Equation 18) (lower is better). Best per dataset in bold. The table highlights the calibration behaviour: BTNR reaches the lowest calibration error on several datasets and stays close to the best elsewhere, which indicates that its predictive intervals have a reliable size.

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
BTNR	0.063 ± 0.005	0.151 ± 0.006	0.104 ± 0.001	0.057 ± 0.001	0.091 ± 0.011	**0.033** ± 0.007	**0.051** ± 0.006	**0.061** ± 0.007	0.066 ± 0.010	0.055 ± 0.001
GP	0.045 ± 0.008	0.160 ± 0.006	0.227 ± 0.010	0.175 ± 0.004	0.131 ± 0.000	0.072 ± 0.000	0.123 ± 0.003	0.160 ± 0.000	0.201 ± 0.001	0.100 ± 0.000
BDE	**0.044** ± 0.001	0.277 ± 0.144	**0.054** ± 0.001	**0.038** ± 0.002	**0.043** ± 0.004	0.040 ± 0.005	0.128 ± 0.005	0.072 ± 0.005	**0.021** ± 0.002	0.109 ± 0.155
HSBNN	0.100 ± 0.002	0.199 ± 0.004	0.120 ± 0.002	0.137 ± 0.008	0.118 ± 0.002	0.174 ± 0.004	0.118 ± 0.007	0.128 ± 0.004	0.125 ± 0.005	**0.049** ± 0.005
BASS	0.186 ± 0.003	**0.140** ± 0.045	0.193 ± 0.003	0.174 ± 0.004	0.168 ± 0.009	0.150 ± 0.010	0.170 ± 0.014	0.139 ± 0.014	0.169 ± 0.002	0.172 ± 0.008

Table 5: Discrimination metrics. The area under the sparsification error (AUSE) (Equation 76) (lower is better) and the Spearman correlation between predicted uncertainty and absolute error (higher is better). Best per dataset within each block is in bold.

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
AUSE										
BTNR	0.955 ± 0.063	0.021 ± 0.003	41.927 ± 0.557	80.769 ± 0.610	2.483 ± 0.109	0.229 ± 0.023	0.404 ± 0.030	2.429 ± 0.165	271.222 ± 2.801	1.256 ± 0.017
GP	0.988 ± 0.036	0.068 ± 0.004	77.639 ± 2.801	69.969 ± 1.046	2.694 ± 0.000	1.390 ± 0.000	0.206 ± 0.009	3.338 ± 0.000	508.673 ± 17.499	1.476 ± 0.000
BDE	**0.603** ± 0.011	0.025 ± 0.003	**15.255** ± 0.334	**7.170** ± 0.086	**1.327** ± 0.045	**0.121** ± 0.011	**0.069** ± 0.019	**1.909** ± 0.130	**39.516** ± 1.967	1.473 ± 0.168
HSBNN	1.212 ± 0.035	**0.011** ± 0.002	57.785 ± 2.843	48.563 ± 8.182	3.252 ± 0.160	2.147 ± 0.391	0.532 ± 0.067	3.999 ± 0.441	252.236 ± 10.867	**1.237** ± 0.065
BASS	1.117 ± 0.146	0.023 ± 0.009	56.245 ± 2.425	33.984 ± 1.401	2.838 ± 0.362	0.278 ± 0.053	0.521 ± 0.100	3.686 ± 0.431	172.940 ± 16.596	1.444 ± 0.108
Spearman										
BTNR	0.171 ± 0.023	0.546 ± 0.056	0.348 ± 0.009	-0.213 ± 0.009	0.222 ± 0.041	0.269 ± 0.125	0.407 ± 0.038	0.107 ± 0.066	-0.067 ± 0.014	**0.182** ± 0.011
GP	0.165 ± 0.016	0.188 ± 0.054	-0.051 ± 0.043	-0.481 ± 0.056	0.274 ± 0.000	0.079 ± 0.000	0.405 ± 0.020	0.019 ± 0.000	-0.530 ± 0.033	0.087 ± 0.000
BDE	**0.443** ± 0.010	**0.938** ± 0.040	**0.653** ± 0.005	**0.662** ± 0.006	**0.439** ± 0.028	**0.594** ± 0.050	**0.911** ± 0.015	**0.226** ± 0.050	**0.697** ± 0.009	0.122 ± 0.321
HSBNN	-0.020 ± 0.020	0.593 ± 0.065	0.185 ± 0.028	0.021 ± 0.090	0.150 ± 0.042	-0.122 ± 0.137	0.040 ± 0.105	-0.108 ± 0.079	0.145 ± 0.032	0.078 ± 0.041
BASS	0.090 ± 0.058	0.714 ± 0.070	0.196 ± 0.025	0.074 ± 0.039	0.075 ± 0.086	0.103 ± 0.151	0.285 ± 0.103	-0.031 ± 0.047	0.187 ± 0.070	-0.069 ± 0.102

Table 6: Outlier detection averaged across the ten datasets. The table reports the outlier detection metrics AUROC and AUPR (Equation 77), under both anomaly scores, averaged across the ten datasets. Because the per dataset values are highly uniform the error bar are small. Models are rows and the four metrics are columns, and each cell is the mean over datasets of the per dataset seed mean plus or minus the standard deviation across datasets. The results show that detection is governed almost entirely by the choice of anomaly score rather than by the model: the per point NLL, which contains the residual term $(y_s - \mu_s)^2/\sigma_s^2$, flags the corrupted targets almost perfectly for every method (AUROC and AUPR near one), whereas the predictive standard deviation alone, which does not see the corrupted target, stays close to chance for all methods.

Model	AUROC (std)	AUPR (std)	AUROC (NLL)	AUPR (NLL)
BTNR	**0.504** ± 0.024	**0.125** ± 0.026	**0.999** ± 0.002	**0.985** ± 0.037
GP	0.488 ± 0.050	0.108 ± 0.015	**0.999** ± 0.002	0.977 ± 0.046
BDE	0.494 ± 0.031	0.115 ± 0.012	0.984 ± 0.035	0.966 ± 0.064
HSBNN	0.494 ± 0.016	0.120 ± 0.021	**0.999** ± 0.002	0.980 ± 0.042
BASS	0.503 ± 0.019	0.121 ± 0.020	0.998 ± 0.003	0.983 ± 0.027

D Expanded Experimental Section

D.1 Hyperparameter ablation

All methods are evaluated across:

- **Seeds**: $\{42, 7, 123, 256, 999\}$ (5 random seeds)
- **Datasets**: concrete, abalone, energy_efficiency, realstate, student_perf, obesity, bike, appliances, ai4i, seoulBike (10 datasets)

Tensor Networks Methods

MPO2, LMPO2, BTT with ALS training:

- $L \in \{3, 4\}$ (polynomial degree of feature map)
- $r \in \{6, 12, 18\}$ (edge dimension)
- $\alpha \in \{1.0, 5.0\}$ (edge prior strength)
- $\sigma_{\text{init}} \in \{0.01, 0.1\}$ (initialization strength)

ALS-CPD (CPD with ALS training)

- $L \in \{3, 4\}$ (polynomial degree of feature map)
- $r \in \{18, 32, 64\}$ (edge dimension)
- $\alpha \in \{1.0, 5.0\}$ (edge prior strength)
- $\sigma_{\text{init}} \in \{0.01, 0.1\}$ (initialization strength)

MPO2, LMPO2, BTT with BTNR training

- $L \in \{3, 4\}$ (polynomial degree of feature map)
- $r = 18$ (edge dimension)
- $\alpha \in \{1.0, 5.0\}$ (edge prior strength)
- $\sigma_{\text{init}} \in \{0.01, 0.1\}$ (initialization strength)

CPD with BTNR training

- $L \in \{3, 4\}$ (polynomial degree of feature map)
- $r = 64$ (edge dimension)
- $\alpha \in \{1.0, 5.0\}$ (edge prior strength)
- $\sigma_{\text{init}} \in \{0.01, 0.1\}$ (initialization strength)

Baseline Methods

We compare against four established probabilistic regression baselines, each providing a predictive mean together with a calibrated predictive standard deviation.

Gaussian Process (GP) (MacKay et al., 1998). A non parametric Bayesian model that places a prior directly over functions through a covariance (kernel) function. The posterior predictive distribution is available in closed form, and the kernel hyperparameters are fitted by maximizing the marginal log likelihood. We use a squared exponential (RBF) or Matérn 5/2 kernel, optionally with automatic relevance determination (per dimension lengthscales). On larger datasets, where exact inference scales cubically in the number of points, we instead employ a sparse variational GP with inducing points. We utilize the gpytorch implementation (Gardner et al., 2018).

Bayesian Deep Ensemble (BDE) (Sommer et al., 2025). We adopt the Microcanonical Langevin Ensembles (MILE) approach, which performs sampling based inference over the weights of a Bayesian neural network. Each ensemble member is initialized by a deterministic optimization warm up and then explored with Microcanonical Langevin Monte Carlo (MCLMC), which bypasses the Metropolis Hastings accept/reject step for faster and more predictable traversal of the multimodal posterior. Uncertainty is obtained by aggregating the posterior predictive samples across the chains. We utilize the implementation in (bde).

Horseshoe Bayesian Neural Network (HSBNN) (Overweg et al., 2019). A single hidden layer Bayesian neural network equipped with a tied, sparsity inducing Horseshoe prior on the input weights, yielding automatic relevance determination and interpretable feature selection. The posterior is approximated by variational inference (Bayes by Backprop layers together with the Horseshoe scale hyperparameters), and the predictive distribution is recovered through Monte Carlo sampling. We utilize the implementation in (Microsoft)

Bayesian Adaptive Smoothing Splines (BASS) (Friedman, 1991). A non parametric Bayesian regression model that represents the response as an adaptive expansion in tensor product spline basis functions, in the spirit of (Bayesian) multivariate adaptive regression splines. The number of basis functions, their interaction order, and their knot locations are inferred jointly through reversible jump Markov chain Monte Carlo, so that model complexity is selected automatically from the data. We use the public pyBASS implementation (Los Alamos National Laboratory, 2021).

BDE (Bayesian Deep Ensemble)

- Hidden layers $\in \{[16, 16], [32, 32]\}$
- Number of ensemble members $M \in \{4, 8\}$

Exact GP (Gaussian Process)

- Kernel type $\in \{\text{RBF}, \text{Matérn}\}$
- ARD $\in \{\text{true}, \text{false}\}$

Note: Only evaluated on smaller datasets (concrete, energy_efficiency, realstate, student_perf).

Sparse GP (Sparse Gaussian Process)

- Number of inducing points $n_{\text{ind}} \in \{50, 100\}$
- Kernel type $\in \{\text{RBF}, \text{Matérn}\}$

Note: Only evaluated on larger datasets (abalone, obesity, bike, appliances, ai4i, seoulBike).

Horseshoe BNN (Bayesian Neural Network)

- Number of hidden units $n_h \in \{32, 64\}$
- Number of posterior samples $n_s \in \{5, 10\}$

BASS

- Maximum interaction order $d_{\max} \in \{2, 3\}$
- Maximum number of basis functions $B_{\max} \in \{30, 50\}$

In Table 7 we report the shorhand notation of the datasets and their size. All data is normalized.

UCI ID	Code	Dataset	Train	Val	Test	Feat.
374	AP	Appliances Energy Prediction	13814	2960	2961	27
275	BK	Bike Sharing	12165	2607	2607	12
601	AI	AI4I	7000	1500	1500	9
560	SB	Seoul Bike Sharing	6132	1314	1314	18
1	AB	Abalone	2923	627	627	11
544	OB	Obesity Levels	1477	317	317	39
165	CO	Concrete Compressive Strength	721	154	155	8
242	EE	Energy Efficiency	537	115	116	8
320	SP	Student Performance	454	97	98	50
477	RS	Real Estate Valuation	289	62	63	6

Table 7: Datasets with shorthand codes used, tasks, number of observations, and number of features.

In Figure 10 we use a one dimensional function to visualize the predictions and uncertainty quantification for each sample point.

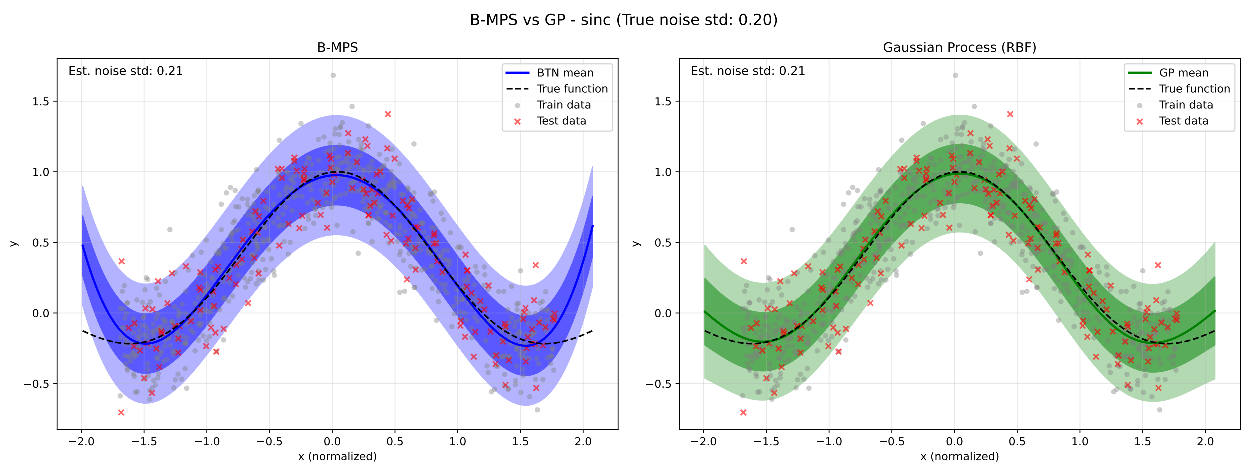

Figure 10: Comparison of prediction and riconstruction error of synthetic data generated using $\sin(\pi x)/\pi x$ and artificially added noise with std 0.2.

In Table 8 we plot the time of an ablation study in a toy scenario, where an ALS model is ablated over all configuration of edge dimensions in a finite set {1,2,3} and then report the best result in the validation set, compared with a run of BTNR.

	ALS	BTNR
sec	2781	9
quality	0.83	0.82

Table 8: LMPO2, third degree polynomial feature map, ablation edge dimension set {1,2,3}.